

RickMorty Explorer

Codelab completo y memoria tecnica de apoyo

Proyecto Android en Java + XML con Retrofit, Gson, RxJava2 y RecyclerView

Asignatura	Computacion Movil
Proyecto	RickMortyExplorer
API REST	Rick and Morty API
Tecnologias	Java, XML, Retrofit, Gson, RxJava2, RecyclerView, Picasso
Fecha	Mayo de 2026

Indice

- 1. Objetivo del proyecto
- 2. Requisitos cubiertos
- 3. API utilizada
- 4. Estructura del proyecto
- 5. Configuración Gradle
- 6. AndroidManifest
- 7. Recursos XML y layouts
- 8. Modelos Java
- 9. Retrofit y endpoints
- 10. RecyclerView: Adapter y ViewHolder
- 11. MainActivity
- 12. DetailActivity
- 13. Flujo completo de la aplicación
- 14. Estados de loading y error
- 15. Errores típicos y solución
- 16. Pruebas realizadas
- 17. Posibles ampliaciones
- 18. Guía de defensa

1. Objetivo del proyecto

El objetivo de este proyecto es desarrollar una aplicacion Android completa, sencilla y defendible que consuma una API REST externa y muestre informacion en una interfaz construida con Java y XML.

Idea principal

RickMorty Explorer muestra una lista de personajes de la Rick and Morty API. Al seleccionar un personaje, se abre una pantalla de detalle con datos ampliados.

2. Requisitos cubiertos

Requisito	Implementacion	Estado
Java + Views/XML	Activities en Java y layouts XML	Cumplido
Retrofit + Gson + RxJava2	RetrofitClient, GsonConverterFactory y RxJava2CallAdapterFactory	Cumplido
RecyclerView	CharacterAdapter con CharacterViewHolder	Cumplido
Endpoint lista	GET /api/character?page=1	Cumplido
Endpoint detalle	GET /api/character/{id}	Cumplido
Loading y error	ProgressBar, contenedor de error y boton de reintento	Cumplido

3. API utilizada

Se utiliza la Rick and Morty API, una API publica sin clave de acceso.

```
Base URL: https://rickandmortyapi.com/api/  
Lista:    GET character?page=1  
Detalle:  GET character/{id}
```

La respuesta de lista contiene informacion de paginacion y un array results. La respuesta de detalle devuelve un unico personaje con campos como id, name, status, species, gender, origin, location e image.

4. Estructura del proyecto

```
com.example.rickmortyexplorer  
|-- model  
|   |-- CharacterItem.java  
|   |-- CharacterListResponse.java  
|   |-- Info.java  
|   |-- Origin.java  
|-- rest  
|   |-- RetrofitClient.java  
|   |-- api  
|       |-- RickMortyApiService.java  
|-- ui  
|   |-- MainActivity.java  
|   |-- DetailActivity.java  
|   |-- adapter  
|       |-- CharacterAdapter.java  
|-- util  
|   |-- CharacterUtils.java
```

Esta separacion ayuda a que cada parte tenga una responsabilidad clara: modelos para datos, rest para red, ui para pantallas, adapter para RecyclerView y util para formateo.

5. Configuración Gradle

El proyecto usa Gradle con Version Catalogs para centralizar versiones.

```
[versions]
agp = "9.0.0"
appcompat = "1.7.0"
recyclerview = "1.4.0"
retrofit = "2.12.0"
rxjava2 = "2.2.21"
rxandroid = "2.1.1"
picasso = "2.8"

[plugins]
android-application = { id = "com.android.application", version.ref = "agp" }

[libraries]
appcompat = { module = "androidx.appcompat:appcompat", version.ref = "appcompat" }
recyclerview = { module = "androidx.recyclerview:recyclerview", version.ref = "recyclerview" }
retrofit = { module = "com.squareup.retrofit2:retrofit", version.ref = "retrofit" }
retrofit-gson = { module = "com.squareup.retrofit2:converter-gson", version.ref = "retrofit" }
retrofit-rxjava2 = { module = "com.squareup.retrofit2:adapter-rxjava2", version.ref = "retrofit" }
rxjava2 = { module = "io.reactivex.rxjava2:rxjava", version.ref = "rxjava2" }
rxandroid2 = { module = "io.reactivex.rxjava2:rxandroid", version.ref = "rxandroid" }
picasso = { module = "com.squareup.picasso:picasso", version.ref = "picasso" }
```

En el módulo app se activa ViewBinding y se declaran las dependencias principales.

```
plugins {
    alias(libs.plugins.android.application)
}

android {
    namespace = "com.example.rickmortyexplorer"
    compileSdk = 36

    defaultConfig {
        applicationId = "com.example.rickmortyexplorer"
        minSdk = 24
        targetSdk = 36
        versionCode = 1
        versionName = "1.0"

        testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"
    }

    buildFeatures {
        viewBinding = true
    }

    buildTypes {
        release {
            isMinifyEnabled = false
            proguardFiles(
                getDefaultProguardFile("proguard-android-optimize.txt"),
                "proguard-rules.pro"
            )
        }
    }

    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_11
        targetCompatibility = JavaVersion.VERSION_11
    }
}

dependencies {
    implementation(libs.appcompat)
    implementation(libs.recyclerview)

    implementation(libs.retrofit)
    implementation(libs.retrofit.gson)
    implementation(libs.retrofit.rxjava2)
    implementation(libs.rxjava2)
    implementation(libs.rxandroid2)
    implementation(libs.picasso)
}
```

6. AndroidManifest

El Manifest declara el permiso de Internet, necesario para consumir la API REST, y registra las dos actividades de la aplicación.

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <uses-permission android:name="android.permission.INTERNET" />

    <application
        android:allowBackup="true"
        android:icon="@drawable/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@drawable/ic_launcher"
        android:supportRtl="true"
        android:theme="@style/Theme.RickMortyExplorer">
        <activity
            android:name=".ui.DetailActivity"
            android:exported="false" />
        <activity
            android:name=".ui.MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

7. Recursos XML y layouts

7.1 Pantalla principal

El layout activity_main.xml contiene una cabecera, un RecyclerView, un ProgressBar y un contenedor de error con botón de reintento.

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/app_bg"
    android:orientation="vertical">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="112dp"
        android:background="@drawable/bg_header"
        android:gravity="bottom"
        android:orientation="vertical"
        android:padding="20dp">

        <TextView
            android:id="@+id/titleText"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="RickMorty Explorer"
            android:textColor="@color/white"
            android:textSize="26sp"
            android:textStyle="bold" />

        <TextView
            android:id="@+id/subtitleText"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="4dp"
            android:text="Personajes de la API pública"
            android:textColor="#DDF8F3"
            android:textSize="15sp" />
    </LinearLayout>

    <FrameLayout
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1">

        <androidx.recyclerview.widget.RecyclerView
```

```

        android:id="@+id/charactersRecyclerView"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:clipToPadding="false"
        android:padding="12dp" />

<ProgressBar
    android:id="@+id/loadingProgress"
    android:layout_width="48dp"
    android:layout_height="48dp"
    android:layout_gravity="center"
    android:visibility="gone" />

<LinearLayout
    android:id="@+id/errorContainer"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_gravity="center"
    android:gravity="center"
    android:orientation="vertical"
    android:padding="24dp"
    android:visibility="gone">

    <TextView
        android:id="@+id/errorText"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:gravity="center"
        android:textColor="@color/app_error"
        android:textSize="16sp"

```

...

7.2 Fila del RecyclerView

La fila row_character.xml muestra imagen, nombre, especie, genero y estado del personaje.

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginBottom="10dp"
    android:background="@drawable/bg_row"
    android:clickable="true"
    android:focusable="true"
    android:foreground="?android:attr/selectableItemBackground"
    android:gravity="center_vertical"
    android:minHeight="92dp"
    android:orientation="horizontal"
    android:padding="10dp">

    <ImageView
        android:id="@+id/characterImage"
        android:layout_width="72dp"
        android:layout_height="72dp"
        android:background="@drawable/bg_image"
        android:contentDescription="@string/app_name"
        android:scaleType="centerCrop" />

    <LinearLayout
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_marginStart="14dp"
        android:layout_weight="1"
        android:orientation="vertical">

        <TextView
            android:id="@+id/nameText"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:ellipsize="end"
            android:maxLines="1"
            android:textColor="@color/app_text"
            android:textSize="18sp"
            android:textStyle="bold" />

        <TextView
            android:id="@+id/speciesText"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="4dp"
            android:textColor="@color/app_muted"
            android:textSize="14sp" />

    <TextView

```

```

        android:id="@+id/statusText"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="2dp"
        android:textColor="@color/app_primary_dark"
        android:textSize="14sp"
        android:textStyle="bold" />
    </LinearLayout>
</LinearLayout>

```

7.3 Pantalla de detalle

El layout `activity_detail.xml` repite el mismo enfoque de estados: contenido, carga y error.

8. Modelos Java

Los modelos son POJOs que coinciden con el JSON devuelto por la API. Gson rellena estos objetos automáticamente.

```

package com.example.rickmortyexplorer.model;

public class CharacterItem {
    private int id;
    private String name;
    private String status;
    private String species;
    private String type;
    private String gender;
    private Origin origin;
    private Origin location;
    private String image;
    private String url;

    public int getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public String getStatus() {
        return status;
    }

    public String getSpecies() {
        return species;
    }

    public String getType() {
        return type;
    }

    public String getGender() {
        return gender;
    }

    public Origin getOrigin() {
        return origin;
    }

    public Origin getLocation() {
        return location;
    }

    public String getImage() {
        return image;
    }

    public String getUrl() {
        return url;
    }
}

package com.example.rickmortyexplorer.model;

import java.util.List;

public class CharacterListResponse {
    private Info info;
    private List<CharacterItem> results;
}

```

```

    public Info getInfo() {
        return info;
    }

    public List<CharacterItem> getResults() {
        return results;
    }
}

```

9. Retrofit y endpoints

RetrofitClient centraliza la URL base y registra Gson y RxJava2. RickMortyApiService define los endpoints mediante anotaciones.

```

package com.example.rickmortyexplorer.rest;

import retrofit2.Retrofit;
import retrofit2.adapter.rxjava2.RxJava2CallAdapterFactory;
import retrofit2.converter.gson.GsonConverterFactory;

public class RetrofitClient {
    private static final String BASE_URL = "https://rickandmortyapi.com/api/";
    private static Retrofit retrofit;

    private RetrofitClient() {
    }

    public static Retrofit getRetrofit() {
        if (retrofit == null) {
            retrofit = new Retrofit.Builder()
                .baseUrl(BASE_URL)
                .addConverterFactory(GsonConverterFactory.create())
                .addCallAdapterFactory(RxJava2CallAdapterFactory.create())
                .build();
        }
        return retrofit;
    }
}

package com.example.rickmortyexplorer.rest.api;

import com.example.rickmortyexplorer.model.CharacterItem;
import com.example.rickmortyexplorer.model.CharacterListResponse;

import io.reactivex.Single;
import retrofit2.http.GET;
import retrofit2.http.Path;
import retrofit2.http.Query;

public interface RickMortyApiService {
    @GET("character")
    Single<CharacterListResponse> getCharacters(@Query("page") int page);

    @GET("character/{id}")
    Single<CharacterItem> getCharacterDetail(@Path("id") int id);
}

```

10. RecyclerView: Adapter y ViewHolder

El adaptador recibe una lista de personajes y se encarga de pintar cada fila. También define una interfaz de click para abrir el detalle.

```

package com.example.rickmortyexplorer.ui.adapter;

import android.view.LayoutInflater;
import android.view.ViewGroup;

import androidx.annotation.NonNull;
import androidx.recyclerview.widget.RecyclerView;

import com.example.rickmortyexplorer.databinding.RowCharacterBinding;
import com.example.rickmortyexplorer.model.CharacterItem;
import com.example.rickmortyexplorer.util.CharacterUtils;
import com.squareup.picasso.Picasso;

import java.util.ArrayList;
import java.util.List;

public class CharacterAdapter extends
    RecyclerView.Adapter<CharacterAdapter.CharacterViewHolder> {

```

```

public interface OnCharacterClickListener {
    void onCharacterClick(CharacterItem character);
}

private final List<CharacterItem> characters = new ArrayList<>();
private final OnCharacterClickListener listener;

public CharacterAdapter(OnCharacterClickListener listener) {
    this.listener = listener;
}

public void submitList(List<CharacterItem> newCharacters) {
    characters.clear();
    if (newCharacters != null) {
        characters.addAll(newCharacters);
    }
    notifyDataSetChanged();
}

@NonNull
@Override
public CharacterViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {
    LayoutInflater inflater = LayoutInflater.from(parent.getContext());
    RowCharacterBinding binding = RowCharacterBinding.inflate(inflater, parent, false);
    return new CharacterViewHolder(binding);
}

@Override
public void onBindViewHolder(@NonNull CharacterViewHolder holder, int position) {
    holder.bind(characters.get(position));
}

@Override
public int getItemCount() {
    return characters.size();
}

class CharacterViewHolder extends RecyclerView.ViewHolder {
    private final RowCharacterBinding binding;

    CharacterViewHolder(RowCharacterBinding binding) {
        super(binding.getRoot());
        this.binding = binding;
    }

    void bind(CharacterItem character) {
        binding.nameText.setText(CharacterUtils.safeText(character.getName()));
        binding.speciesText.setText(CharacterUtils.formatSubtitle(character));
        binding.statusText.setText(CharacterUtils.formatStatus(character));

        Picasso.get()
            .load(character.getImage())
            .fit()
            .centerCrop()
            .into(binding.characterImage);

        binding.getRoot().setOnClickListener(view ->
            listener.onCharacterClick(character));
    }
}

```

11. MainActivity

La pantalla principal crea el servicio Retrofit, configura el RecyclerView y llama al endpoint de lista. La petición se ejecuta en segundo plano con `Schedulers.io()` y vuelve al hilo principal con `AndroidSchedulers.mainThread()`.

```

package com.example.rickmortyexplorer.ui;

import android.content.Intent;
import android.os.Bundle;
import android.view.View;

import androidx.appcompat.app.AppCompatActivity;
import androidx.recyclerview.widget.LinearLayoutManager;

import com.example.rickmortyexplorer.databinding.ActivityMainBinding;
import com.example.rickmortyexplorer.rest.RetrofitClient;
import com.example.rickmortyexplorer.rest.api.RickMortyApiService;
import com.example.rickmortyexplorer.ui.adapter.CharacterAdapter;

```

```

import io.reactivex.android.schedulers.AndroidSchedulers;
import io.reactivex.disposables.CompositeDisposable;
import io.reactivex.schedulers.Schedulers;

public class MainActivity extends AppCompatActivity {
    private ActivityMainBinding binding;
    private RickMortyApiService apiService;
    private CharacterAdapter adapter;
    private final CompositeDisposable compositeDisposable = new CompositeDisposable();

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        binding = ActivityMainBinding.inflate(getLayoutInflater());
        setContentView(binding.getRoot());

        apiService = RetrofitClient.getRetrofit().create(RickMortyApiService.class);
        adapter = new CharacterAdapter(character -> {
            Intent intent = new Intent(this, DetailActivity.class);
            intent.putExtra(DetailActivity.EXTRA_CHARACTER_ID, character.getId());
            startActivity(intent);
        });

        binding.charactersRecyclerView.setLayoutManager(new LinearLayoutManager(this));
        binding.charactersRecyclerView.setAdapter(adapter);
        binding.retryButton.setOnClickListener(view -> loadCharacters());

        loadCharacters();
    }

    private void loadCharacters() {
        showLoading();
        compositeDisposable.add(
            apiService.getCharacters(1)
                .subscribeOn(Schedulers.io())
                .observeOn(AndroidSchedulers.mainThread())
                .subscribe(
                    response -> {
                        adapter.submitList(response.getResults());
                        showContent();
                    },
                    throwable -> showError("No se pudieron cargar los
personajes.")
                )
        );
    }

    private void showLoading() {
        binding.loadingProgress.setVisibility(View.VISIBLE);
        binding.errorContainer.setVisibility(View.GONE);
        binding.charactersRecyclerView.setVisibility(View.GONE);
    }

    private void showContent() {
        binding.loadingProgress.setVisibility(View.GONE);
        binding.errorContainer.setVisibility(View.GONE);
        binding.charactersRecyclerView.setVisibility(View.VISIBLE);
    }

    private void showError(String message) {
        binding.loadingProgress.setVisibility(View.GONE);
        binding.errorText.setText(message);
        binding.errorContainer.setVisibility(View.VISIBLE);
        binding.charactersRecyclerView.setVisibility(View.GONE);
    }

    @Override
    protected void onDestroy() {
        super.onDestroy();
        compositeDisposable.clear();
    }
}

```

12. DetailActivity

La pantalla de detalle recibe el id del personaje por Intent y consume el endpoint de detalle.

```

package com.example.rickmortyexplorer.ui;

import android.os.Bundle;
import android.view.View;

import androidx.appcompat.app.AppCompatActivity;

```

```

import com.example.rickmorthyexplorer.databinding.ActivityDetailBinding;
import com.example.rickmorthyexplorer.model.CharacterItem;
import com.example.rickmorthyexplorer.rest.RetrofitClient;
import com.example.rickmorthyexplorer.rest.api.RickMortyApiService;
import com.example.rickmorthyexplorer.util.CharacterUtils;
import com.squareup.picasso.Picasso;

import io.reactivex.android.schedulers.AndroidSchedulers;
import io.reactivex.disposables.CompositeDisposable;
import io.reactivex.schedulers.Schedulers;

public class DetailActivity extends AppCompatActivity {
    public static final String EXTRA_CHARACTER_ID = "extra_character_id";

    private ActivityDetailBinding binding;
    private RickMortyApiService apiService;
    private final CompositeDisposable compositeDisposable = new CompositeDisposable();
    private int characterId;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        binding = ActivityDetailBinding.inflate(getLayoutInflater());
        setContentView(binding.getRoot());

        apiService = RetrofitClient.getRetrofit().create(RickMortyApiService.class);
        characterId = getIntent().getIntExtra(EXTRA_CHARACTER_ID, -1);

        binding.backButton.setOnClickListener(view -> finish());
        binding.detailRetryButton.setOnClickListener(view -> loadCharacterDetail());

        loadCharacterDetail();
    }

    private void loadCharacterDetail() {
        if (characterId <= 0) {
            showError("No se encontro el personaje seleccionado.");
            return;
        }

        showLoading();
        compositeDisposable.add(
            apiService.getCharacterDetail(characterId)
                .subscribeOn(Schedulers.io())
                .observeOn(AndroidSchedulers.mainThread())
                .subscribe(
                    this::showCharacter,
                    throwable -> showError("No se pudo cargar el detalle.")
                )
        );
    }

    private void showCharacter(CharacterItem character) {
        binding.detailToolBarTitle.setText(CharacterUtils.safeText(character.getName()));
        binding.detailNameText.setText(CharacterUtils.safeText(character.getName()));
        binding.detailStatusText.setText(CharacterUtils.formatStatus(character));
        binding.detailInfoText.setText(CharacterUtils.formatDetail(character));

        Picasso.get()
            .load(character.getImage())
            .fit()
            .centerCrop()
            .into(binding.detailImage);

        binding.detailLoadingProgress.setVisibility(View.GONE);
        binding.detailErrorContainer.setVisibility(View.GONE);
        binding.contentContainer.setVisibility(View.VISIBLE);
    }

    private void showLoading() {
        binding.detailLoadingProgress.setVisibility(View.VISIBLE);
        binding.detailErrorContainer.setVisibility(View.GONE);
        binding.contentContainer.setVisibility(View.GONE);
    }

    private void showError(String message) {
        binding.detailLoadingProgress.setVisibility(View.GONE);
        binding.detailErrorText.setText(message);
        binding.detailErrorContainer.setVisibility(View.VISIBLE);
        binding.contentContainer.setVisibility(View.GONE);
    }

    @Override
    protected void onDestroy() {
        super.onDestroy();
    }

```

```

        compositeDisposable.clear();
    }
}

```

13. Flujo completo de la aplicacion

- El usuario abre la app.
- MainActivity muestra un ProgressBar.
- Se llama a GET character?page=1.
- Cuando llegan los datos, el RecyclerView muestra los personajes.
- El usuario pulsa un personaje.
- DetailActivity recibe el id por Intent.
- Se llama a GET character/{id}.
- La pantalla de detalle muestra la informacion ampliada.

14. Estados de loading y error

La app no deja la pantalla vacia durante las peticiones. Tanto la lista como el detalle tienen funciones para mostrar carga, contenido o error.

```

showLoading()  -> muestra ProgressBar y oculta contenido/error
showContent()  -> muestra la informacion cargada
showError()    -> muestra mensaje de error y boton de reintento

```

15. Errores tipicos y solucion

Problema	Solucion
La app no tiene Internet	Comprobar el permiso INTERNET en AndroidManifest.xml.
No aparecen los bindings	Verificar que viewBinding = true y que los ids existen en XML.
Error de Gradle	Sincronizar proyecto y revisar versiones de Gradle/AGP.
No carga la API	Comprobar conexion del emulador y URL base de Retrofit.
No abre detalle	Comprobar que se envia EXTRA_CHARACTER_ID en el Intent.

16. Pruebas realizadas

- Sincronizacion Gradle correcta.
- Compilacion debug correcta.
- Ejecucion en emulador Android.
- Carga de lista de personajes.
- Scroll en RecyclerView.
- Apertura de pantalla de detalle al pulsar un personaje.

17. Posibles ampliaciones

- Anadir buscador por nombre.
- Anadir paginacion para cargar mas personajes.
- Guardar favoritos localmente.

- Migrar a MVVM.
- Usar fragments para navegacion adaptable.
- Mostrar localizaciones o episodios como pantallas adicionales.

18. Guia de defensa

Para defender el proyecto se recomienda explicar el flujo con este orden:

- Primero, explicar la API y los dos endpoints usados.
- Despues, enseñar RetrofitClient y RickMortyApiService.
- A continuacion, mostrar MainActivity y el RecyclerView.
- Luego, abrir CharacterAdapter para identificar Adapter y ViewHolder.
- Por ultimo, mostrar DetailActivity y explicar el segundo endpoint.

Resumen final

RickMorty Explorer cumple los requisitos minimos del proyecto final y queda preparado para ampliaciones opcionales sin cambiar la base arquitectonica.